Logistics Application - Project Documentation

Spring Boot 4.1.1 · Java 21 · MySQL

About the project

The Logistics Application is a Spring Boot project that manages the daily work of a logistics company. It keeps the details of customers, warehouses, the products stored in each warehouse, the shipments sent for customers, the tracking updates of every shipment, and the final delivery. All of this is offered as a REST API, so any front end or tool such as Postman or Swagger can use it.

The project is built in layers. Each layer has one job, which keeps the code simple to read and easy to change later.

Technologies used

The project is written in Java 21 with Spring Boot 4.1.1. Spring Data JPA with Hibernate talks to the MySQL database, and the database name is logisticsdb. Spring Security protects the APIs, Swagger (springdoc) shows the APIs in the browser, and Actuator shows the health of the application. Lombok is used to avoid writing getters, setters and constructors by hand. Maven manages all the libraries.

Project structure

All code is inside the package com.logisticsapp, and every sub-package has one clear purpose.

The controller package holds the classes that receive the requests. The service package holds the interfaces that list the operations, and service/impl holds the classes that do the real work. The repository package holds the interfaces that talk to the database. The entity package holds the classes that become database tables, and the enums package holds the shipment status. The dto package has two parts: request for the data coming in and response for the data going out. The mapper package converts between entities and DTOs.

The exception package holds the custom errors and the handler that turns them into clean responses. The config package holds the settings for security, Swagger, caching and auditing. The scheduler package holds the timed task, and the util package holds a small helper for paging.

The data and how it is connected

There are six tables: customers, warehouses, inventory, shipments, tracking and delivery. They are created automatically when the application starts.

One customer can have many shipments, and one warehouse can have many inventory items. One shipment can have many tracking updates, but only one delivery. These links are written in the entity classes with @OneToMany, @ManyToOne and @OneToOne. Every entity also extends the Auditable class, which adds four extra columns for audit details, explained later.

The shipment status is an enum called ShipmentStatus with five values: PENDING, IN_TRANSIT, OUT_FOR_DELIVERY, DELIVERED and CANCELLED.

How a request travels

When a request comes in, the controller receives it and checks the data with @Valid. The controller then calls the service interface, and the service implementation applies the business rules. The service uses the mapper to convert the DTO into an entity, and the repository to save or read it from MySQL.

The result is converted back to a response DTO and sent out inside a ResponseEntity with the proper status code. If anything goes wrong at any step, the global exception handler sends back a clean error message.

Main parts of the application

DTO and mapper classes

Every module has a request DTO and a response DTO, for example CustomerRequestDTO and CustomerResponseDTO. The request DTO carries the data from the user along with the validation rules, and the response DTO sends back only the fields we want to show. The mapper class of each module, for example CustomerMapper, has three methods: toEntity to create a new entity, updateEntity to change an existing one, and toResponse to build the response.

Validation

The rules are written on the request DTO fields using @NotBlank, @NotNull, @Email, @Pattern and @Min. For example, a customer phone number must be exactly 10 digits and a warehouse capacity must be at least 1. The controllers use @Valid, so wrong data is stopped before it reaches the service, and the user gets a clear message for every wrong field.

Create and update

Each module has a POST API to create a record and a PUT API to update it. The service checks the rules before saving: a customer email must be unique, an inventory item cannot push a warehouse above its capacity, and a shipment can have only one delivery.

Exception handling

Two custom exceptions are used: ResourceNotFoundException when a record is not found (404), and BadRequestException when the data or the business rule is wrong (400). The GlobalExceptionHandler class, marked with @RestControllerAdvice, catches these along with validation errors and bad input, and returns a simple JSON message through the ErrorResponse class. It also gives a clear message for a link that does not exist (404), a wrong HTTP method (405), and a file upload that is too big or is not sent as form-data (400). Any other unexpected error returns a safe 500 message.

Pagination and sorting

Every list API returns data page by page. The PageUtil class builds the page request from four values: page, size, sortBy and direction. For example, the user can ask for the second page of five customers sorted by name in descending order. PageUtil also checks the values: the page number cannot be negative and the page size must be between 1 and 100, otherwise a 400 message is returned. The AppConfig class keeps the page output clean and simple.

Transaction

The service implementation classes use @Transactional for create, update and delete. This matters most in two places. When a tracking update is added, the tracking entry is saved and the shipment status is changed in one transaction. When a delivery is recorded, the delivery is saved and the shipment is marked DELIVERED in one transaction. If one step fails, both are cancelled, so the data never stays half done.

Auditing

The Auditable base class adds four columns to every table: createdAt, lastModifiedAt, createdBy and lastModifiedBy. They are filled automatically. The AppConfig class turns auditing on and tells it who made the change, which is the logged-in username (admin or staff), or system if nobody is logged in.

Logging

The service classes, the scheduler and the exception handler use @Slf4j. Important actions, such as a customer being created or a shipment being deleted, are written to the log with log.info, and unexpected errors are written with log.error.

Swagger

The springdoc library creates a Swagger page at /swagger-ui.html where every API can be seen and tested in the browser. The OpenAPIConfig class sets the title and adds an Authorize button for the login, and the controllers use @Tag and @Operation to describe each API.

JPQL and native queries

Besides the ready-made methods of JpaRepository, three custom queries are used. The method findByStatusJpql in ShipmentRepository is a JPQL query that returns the shipments of one status. The method sumQuantityByWarehouse in InventoryRepository is a JPQL query that adds up the stock of a warehouse for the capacity check. The method findLowStock in InventoryRepository is a native SQL query that returns the items whose quantity is below a given number.

Actuator

The Actuator library shows the health and details of the running application. The health, info, metrics and caches endpoints are switched on in application.properties. The health link is open to everyone but shows only the status (UP or DOWN), and the full details are shown only to logged-in users. The other links need a login.

Scheduler

The ShipmentScheduler class runs a task automatically every 30 minutes, at minute 0 and minute 30 of every hour, using @Scheduled with a cron time. It counts the pending and in-transit shipments and writes the numbers to the log. A cron schedule never repeats the runs that were missed, so the log stays clean even if the computer goes to sleep. The annotation @EnableScheduling in AppConfig turns this on.

Connection pool

Spring Boot uses HikariCP as the connection pool, and it is set in application.properties. The pool is named LogisticsPool, it keeps at most 10 connections and at least 2 ready, and it has settings for the connection and idle timeouts. A pool means the application reuses database connections instead of opening a new one for every request, which makes it faster.

Cache

Caching is switched on with @EnableCaching in AppConfig and used in CustomerServiceImpl. Reading one customer by id uses @Cacheable, so a repeated request is answered from memory without going to the database. Updating a customer uses @CachePut to refresh the saved copy, and deleting uses @CacheEvict to remove it. The cache is named customers.

Security

The SecurityConfig class protects the application with HTTP Basic login. There are two users kept in memory, admin and staff, and their passwords are stored in BCrypt form. The two passwords are read from application.properties (app.security.admin-password and app.security.staff-password), so they are not written inside the Java code. Everyone who logs in can create, read and update, but only admin can delete. Swagger and the health link are open without a login. If a staff user tries to delete, a clear 403 message is returned.

File upload

The FileController class lets the user upload a screenshot or a document and view it later. The POST link /api/files saves a png, jpg, jpeg, pdf, doc, docx or txt file of up to 5 MB in the uploads folder of the project and returns the saved file name. The GET link /api/files/{fileName} shows the file: images and pdf files open in the response, and Word files download. The class checks that the file is not empty, that the type is allowed, and that the link cannot reach outside the uploads folder.

Business rules

A few rules keep the data correct. A customer email cannot be used twice. A warehouse cannot hold more than its capacity. A shipment can have only one delivery, and the status of a delivery must be DELIVERED. A new shipment always starts as PENDING, whatever status is sent.

A shipment status can change only in the allowed order. PENDING can go to IN_TRANSIT or CANCELLED. IN_TRANSIT can go to OUT_FOR_DELIVERY, DELIVERED or CANCELLED. OUT_FOR_DELIVERY can go to DELIVERED or CANCELLED. DELIVERED and CANCELLED cannot change. Because of this, a delivery cannot be recorded while the shipment is still PENDING.

A tracking entry can change only its location. To change the status of a shipment, a new tracking update is added, and the shipment follows that status.

Settings

All settings are in the application.properties file. The application runs on port 8081. It connects to the MySQL database logisticsdb on localhost port 3306, and the database is created automatically if it does not exist. The setting ddl-auto=update lets Hibernate create and update the tables, and show-sql prints the SQL in the console. The connection pool, file upload (maximum size 5 MB and the uploads folder), actuator, login passwords and logging settings are in the same file. The database username and password must be changed to your own.

Testing

The project has unit tests written with JUnit and Mockito. CustomerServiceTest checks the customer business logic using a fake repository, so no database is needed. CustomerControllerTest checks the customer endpoint using MockMvc with a fake service. LogisticsApplicationTests is the starter test that checks the application starts, and it needs MySQL to be running.

How to run

1. Start MySQL and set your username and password in application.properties.
2. Open the project in Eclipse and run LogisticsApplication.java as a Java application.
3. Wait for the message that Tomcat started on port 8081.
4. Open http://localhost:8081/swagger-ui.html, click Authorize, and log in as admin to try the APIs.